

Add Constexpr Modifiers to Functions in <algorithm> and <utility> Headers

Changes to [P0202R1](#) are marked with blue. Show/Hide deleted lines from P0202R1

I. Introduction and Motivation

The Standard Library provides a great collection of algorithms, many of which currently lack constexpr support. Even a simple constexpr usage requires reimplementing a big bunch of the Standard Library. Consider the simple example:

```
#include <array>
#include <algorithm>

int main() {
    // OK
    constexpr std::array<char, 6> a { 'H', 'e', 'l', 'l', 'o' };

    // Failures:
    // * std::find is not constexpr
    constexpr auto it = std::find(a.rbegin(), a.rend(), 'H');
}
```

This proposal concentrates on constexpr algorithms, deferring simple containers and iterators to a separate proposal.

A proof of concept implementation for some algorithms, is available at: [rhalbersma](#) and [Boost.Algorithm](#).

II. Impact on the Standard

This proposal is a pure library extension. It proposes changes to existing headers <utility> and <algorithm> such that the changes do not break existing code and do not degrade performance. It does not require any changes in the core language in simple cases of non assembly optimized Standard Library, and it could be implemented in standard C++.

Depending on the Standard Library implementation this proposal may rely on [P0031R0](#). P0031R0 was adopted. P0031R0 provides constexpr additions to std::advance, std::distance, std::move_iterator and other functions and classes. Those may be used by some implementations of <algorithm> header.

III. Design Decisions

A. <cstring> must not have constexpr additions

Existing implementations of the functions in <algorithm> header usually rely on functions from <cstring>. For example std::copy usually takes advantage of std::memmove for POD types. During the Jacksonville meeting it was decided not to modify the <cstring> headers, leading to a decision to use compiler specific intrinsics instead of functions from <cstring> header.

B. Assumption that it is possible to implement all the proposed changes without affecting language core, especially [expr.const]

There are many Standard Library implementations nowadays, including some proprietary. It is impossible to investigate all of them to be 100% sure that no performance degradation possible.

This proposal assumes that:

- If algorithm uses compiler intrinsics, then those intrinsics could be made constexpr by compiler vendors.
- If algorithm uses assembly optimization, then that assembly could be turned into constexpr compiler intrinsic.
- If algorithm uses external functions, then those functions could be made inline and marked constexpr or could be replaced with intrinsics.
- Modern compilers are good in code optimization, so a decently small amount of algorithms use assembly or intrinsics.

C. Analysis of existing <algorithm> implementations.

libstdc++ and libc++ implement <algorithm> differently. libc++ uses some functions from <cstring> header, libstdc++ uses compiler specific intrinsics:

libstdc++	libc++	Some of the Algorithms
__builtin_memmove	std::memmove	copy, sort, partition, copy_backward
__builtin_memset	std::memset	fill, fill_n
__builtin_memcmp		equal, lexicographical_compare

GCC's intrinsic __builtin_memcmp is already usable in constant expressions; intrinsics __builtin_memmove, __builtin_memset could be probably easily tuned to be usable in constant expressions. libc++ will probably have to follow the GCC steps and use intrinsics for std::memmove, std::memset or just remove their usage and rely on compiler's optimizations.

Algorithms stable_partition, inplace_merge and stable_sort allocate memory, construct variables using placement new, use unique_ptr and do other things not acceptable in constexpr expressions. Making those algorithms constexpr seems to be a hard task that would require a lot of intrinsics. Those algorithms are not marked with constexpr in this wording.

Algorithms shuffle and sample rely upon uniform_int_distribution that has no constexpr functions. Those algorithms are not marked with constexpr in this wording.

libc++ uses goto in some algorithms, this must be pretty simple to fix without affecting performance.

D. Do not mark `ExecutionPolicy` and overloads with `constexpr`.

It seems that N4687 accidentally marks some of the `ExecutionPolicy` and overloads with `constexpr`. This wording does not mark the `ExecutionPolicy` and overloads with `constexpr`.

IV. Proposed wording relative to N4594

All the additions to the Standard are marked with underlined green.

A. Modifications to "Header `<algorithm>` synopsis" [`algorithms.general`]

Note for editor: All the functions in [`algorithms.general`] must be marked with `constexpr`, except functions `shuffle`, `sample`, `stable_sort`, `stable_partition`, `inplace_merge` and functions accepting `ExecutionPolicy`.

```
#include <initializer_list>

namespace std {

    // 28.5, non-modifying sequence operations:
    // 28.5.1, all of
    template <class InputIterator, class Predicate>
    constexpr bool all_of(InputIterator first, InputIterator last, Predicate pred);

    template <class ExecutionPolicy, class ForwardIterator, class Predicate>
    bool all_of(ExecutionPolicy&& exec, // see 28.4.5
               ForwardIterator first, ForwardIterator last, Predicate pred);

    // 28.5.2, any of
    template <class InputIterator, class Predicate>
    constexpr bool any_of(InputIterator first, InputIterator last, Predicate pred);

    template <class ExecutionPolicy, class ForwardIterator, class Predicate>
    bool any_of(ExecutionPolicy&& exec, // see 28.4.5
               ForwardIterator first, ForwardIterator last, Predicate pred);

    // 28.5.3, none of
    template <class InputIterator, class Predicate>
    constexpr bool none_of(InputIterator first, InputIterator last, Predicate pred);

    template <class ExecutionPolicy, class ForwardIterator, class Predicate>
    bool none_of(ExecutionPolicy&& exec, // see 28.4.5
               ForwardIterator first, ForwardIterator last, Predicate pred);

    // 28.5.4, for each
    template <class InputIterator, class Function>
    constexpr Function for_each(InputIterator first, InputIterator last, Function f);

    template <class ExecutionPolicy, class ForwardIterator, class Function>
    void for_each(ExecutionPolicy&& exec,
                 ForwardIterator first, ForwardIterator last, Function f);

    template <class InputIterator, class Size, class Function>
    constexpr InputIterator for_each_n(InputIterator first, Size n, Function f);

    template <class ExecutionPolicy, class ForwardIterator, class Size, class Function>
    ForwardIterator for_each_n(ExecutionPolicy&& exec, ForwardIterator first, Size n, Function f);

    // 28.5.5, find
    template<class InputIterator, class T>
    constexpr InputIterator find(InputIterator first, InputIterator last, const T& value);

    template <class ExecutionPolicy, class ForwardIterator, class T>
    ForwardIterator find(ExecutionPolicy&& exec, // see 28.4.5
                       ForwardIterator first, ForwardIterator last,
                       const T& value);

    template <class InputIterator, class Predicate>
    constexpr InputIterator find_if(InputIterator first, InputIterator last, Predicate pred);

    template <class ExecutionPolicy, class ForwardIterator, class Predicate>
    ForwardIterator find_if(ExecutionPolicy&& exec, // see 28.4.5
                          ForwardIterator first, ForwardIterator last,
                          Predicate pred);

    template <class InputIterator, class Predicate>
    constexpr InputIterator find_if_not(InputIterator first, InputIterator last, Predicate pred);

    template <class ExecutionPolicy, class ForwardIterator, class Predicate>
    ForwardIterator find_if_not(ExecutionPolicy&& exec, // see 28.4.5
                              ForwardIterator first, ForwardIterator last,
                              Predicate pred);

    // 28.5.6, find end
    template<class ForwardIterator1, class ForwardIterator2>
    constexpr ForwardIterator1 find_end(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, ForwardIterator2 last2);

    template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
    constexpr ForwardIterator1 find_end(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, ForwardIterator2 last2, BinaryPredicate pred);

    template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
    ForwardIterator1 find_end(ExecutionPolicy&& exec, // see 28.4.5
                            ForwardIterator1 first1, ForwardIterator1 last1,
                            ForwardIterator2 first2, ForwardIterator2 last2);

    template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
    ForwardIterator1 find_end(ExecutionPolicy&& exec, // see 28.4.5
                            ForwardIterator1 first1, ForwardIterator1 last1,
                            ForwardIterator2 first2, ForwardIterator2 last2,
                            BinaryPredicate pred);

    // 28.5.7, find first
    template<class InputIterator, class ForwardIterator>
    constexpr InputIterator find_first_of(InputIterator first1, InputIterator last1, ForwardIterator first2, ForwardIterator last2);

    template<class InputIterator, class ForwardIterator, class BinaryPredicate>
    constexpr InputIterator find_first_of(InputIterator first1, InputIterator last1, ForwardIterator first2, ForwardIterator last2, BinaryPredicate pred);

    template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
```

```

ForwardIterator1 find_first_of(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
ForwardIterator find_first_of(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2,
    BinaryPredicate pred);

// 28.5.8, adjacent find
template<class ForwardIterator>
constexpr ForwardIterator adjacent_find(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class BinaryPredicate>
constexpr ForwardIterator adjacent_find(ForwardIterator first, ForwardIterator last, BinaryPredicate pred);

template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator adjacent_find(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first,
    ForwardIterator last);

template<class ExecutionPolicy, class ForwardIterator, class BinaryPredicate>
ForwardIterator adjacent_find(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first,
    ForwardIterator last,
    BinaryPredicate pred);

// 28.5.9, count
template<class InputIterator, class T>
constexpr typename iterator_traits<InputIterator>::difference_type count(InputIterator first, InputIterator last, const T& value);

template<class ExecutionPolicy, class ForwardIterator, class T>
typename iterator_traits<ForwardIterator>::difference_type count(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last, const T& value);

template<class InputIterator, class Predicate>
constexpr typename iterator_traits<InputIterator>::difference_type count_if(InputIterator first, InputIterator last, Predicate pred);

template<class ExecutionPolicy, class ForwardIterator, class Predicate>
typename iterator_traits<ForwardIterator>::difference_type count_if(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last, Predicate pred);

// 28.5.10, mismatch
template<class InputIterator1, class InputIterator2>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2);

template <class InputIterator1, class InputIterator2, class BinaryPredicate>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2);

template <class InputIterator1, class InputIterator2, class BinaryPredicate>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, BinaryPredicate pred);

template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
pair <ForwardIterator1, ForwardIterator2>
mismatch(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2);

template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
pair <ForwardIterator1, ForwardIterator2>
mismatch(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, BinaryPredicate pred);

template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
pair <ForwardIterator1, ForwardIterator2>
mismatch(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2);

template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
pair <ForwardIterator1, ForwardIterator2>
mismatch(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2,
    BinaryPredicate pred);

// 28.5.11, equal
template<class InputIterator1, class InputIterator2>
constexpr bool equal(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2);

template <class InputIterator1, class InputIterator2, class BinaryPredicate>
constexpr bool equal(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
constexpr bool equal(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2);

template <class InputIterator1, class InputIterator2, class BinaryPredicate>
constexpr bool equal(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, BinaryPredicate pred);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
bool equal(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
bool equal(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, BinaryPredicate pred);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
bool equal(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,

```

```

ForwardIterator2 first2, ForwardIterator2 last2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
bool equal(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2,
    BinaryPredicate pred);

// 28.5.12, is permutation
template<class ForwardIterator1, class ForwardIterator2>
constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2);

template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, BinaryPredicate pred);

template<class ForwardIterator1, class ForwardIterator2>
constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, ForwardIterator2 last2);

template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, ForwardIterator2 last2, BinaryPredicate pred);

// 28.5.13, search
template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator1 search(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, ForwardIterator2 last2);

template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
constexpr ForwardIterator1 search(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, ForwardIterator2 last2, BinaryPredicate pred);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator1 search(
    ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
ForwardIterator1 search(
    ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2,
    BinaryPredicate pred);

template<class ForwardIterator, class Size, class T>
constexpr ForwardIterator search_n(ForwardIterator first, ForwardIterator last, Size count, const T& value);

template<class ForwardIterator, class Size, class T, class BinaryPredicate>
constexpr ForwardIterator search_n(ForwardIterator first, ForwardIterator last, Size count, const T& value, BinaryPredicate pred);

template<class ExecutionPolicy, class ForwardIterator, class Size, class T>
ForwardIterator search_n(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    Size count, const T& value);

template<class ExecutionPolicy, class ForwardIterator, class Size, class T, class BinaryPredicate>
ForwardIterator search_n(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    Size count, const T& value,
    BinaryPredicate pred);

template<class ForwardIterator, class Searcher>
constexpr ForwardIterator search(ForwardIterator first, ForwardIterator last,
    const Searcher &searcher);

// 28.6, modifying sequence operations:
// 28.6.1, copy:
template<class InputIterator, class OutputIterator>
constexpr OutputIterator copy(InputIterator first, InputIterator last, OutputIterator result);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 copy(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first, ForwardIterator1 last,
    ForwardIterator2 result);

template<class InputIterator, class Size, class OutputIterator>
constexpr OutputIterator copy_n(InputIterator first, Size n, OutputIterator result);

template<class ExecutionPolicy, class ForwardIterator1, class Size, class ForwardIterator2>
ForwardIterator2 copy_n(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first, Size n,
    ForwardIterator2 result);

template<class InputIterator, class OutputIterator, class Predicate>
constexpr OutputIterator copy_if(InputIterator first, InputIterator last, OutputIterator result, Predicate pred);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class Predicate>
ForwardIterator2 copy_if(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first, ForwardIterator1 last,
    ForwardIterator2 result, Predicate pred);

template<class BidirectionalIterator1, class BidirectionalIterator2>
constexpr BidirectionalIterator2 copy_backward(BidirectionalIterator1 first, BidirectionalIterator1 last, BidirectionalIterator2 result);

// 28.6.2, move
template<class InputIterator, class OutputIterator>
constexpr OutputIterator move(InputIterator first, InputIterator last, OutputIterator result);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 move(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first, ForwardIterator1 last,
    ForwardIterator2 result);

template<class BidirectionalIterator1, class BidirectionalIterator2>
constexpr BidirectionalIterator2 move_backward(BidirectionalIterator1 first, BidirectionalIterator1 last, BidirectionalIterator2 result);

// 28.6.3, swap
template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator2 swap_ranges(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 swap_ranges(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,

```

```

        ForwardIterator2 first2);

template<class ForwardIterator1, class ForwardIterator2>
constexpr void iter_swap(ForwardIterator1 a, ForwardIterator2 b);

// 28.6.4, transform
template<class InputIterator, class OutputIterator, class UnaryOperation>
constexpr OutputIterator transform(InputIterator first, InputIterator last, OutputIterator result, UnaryOperation op);

template<class InputIterator1, class InputIterator2, class OutputIterator, class BinaryOperation>
constexpr OutputIterator transform(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, OutputIterator result, BinaryOperation binary_op);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class UnaryOperation>
ForwardIterator2 transform(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first, ForwardIterator1 last,
    ForwardIterator2 result, UnaryOperation op);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class ForwardIterator, class BinaryOperation>
ForwardIterator transform(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator result,
    BinaryOperation binary_op);

// 28.6.5, replace
template<class ForwardIterator, class T>
constexpr void replace(ForwardIterator first, ForwardIterator last, const T& old_value, const T& new_value);

template<class ExecutionPolicy, class ForwardIterator, class T>
void replace(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    const T& old_value, const T& new_value);

template<class ForwardIterator, class Predicate, class T>
constexpr void replace_if(ForwardIterator first, ForwardIterator last, Predicate pred, const T& new_value);

template<class ExecutionPolicy, class ForwardIterator, class Predicate, class T>
void replace_if(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    Predicate pred, const T& new_value);

template<class InputIterator, class OutputIterator, class T>
constexpr OutputIterator replace_copy(InputIterator first, InputIterator last, OutputIterator result, const T& old_value, const T& new_value);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class T>
ForwardIterator2 replace_copy(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first, ForwardIterator1 last,
    ForwardIterator2 result,
    const T& old_value, const T& new_value);

template<class InputIterator, class OutputIterator, class Predicate, class T>
constexpr OutputIterator replace_copy_if(InputIterator first, InputIterator last, OutputIterator result, Predicate pred, const T& new_value);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class Predicate, class T>
ForwardIterator2 replace_copy_if(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first, ForwardIterator1 last,
    ForwardIterator2 result,
    Predicate pred, const T& new_value);

// 28.6.6, fill
template<class ForwardIterator, class T>
constexpr void fill(ForwardIterator first, ForwardIterator last, const T& value);

template<class ExecutionPolicy, class ForwardIterator, class T>
void fill(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last, const T& value);

template<class OutputIterator, class Size, class T>
constexpr OutputIterator fill_n(OutputIterator first, Size n, const T& value);

template<class ExecutionPolicy, class ForwardIterator, class Size, class T>
ForwardIterator fill_n(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, Size n, const T& value);

// 28.6.7, generate
template<class ForwardIterator, class Generator>
constexpr void generate(ForwardIterator first, ForwardIterator last, Generator gen);

template<class ExecutionPolicy, class ForwardIterator, class Generator>
void generate(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    Generator gen);

template<class OutputIterator, class Size, class Generator>
constexpr OutputIterator generate_n(OutputIterator first, Size n, Generator gen);

template<class ExecutionPolicy, class ForwardIterator, class Size, class Generator>
ForwardIterator generate_n(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, Size n, Generator gen);

// 28.6.8, remove
template<class ForwardIterator, class T>
constexpr ForwardIterator remove(ForwardIterator first, ForwardIterator last, const T& value);

template<class ExecutionPolicy, class ForwardIterator, class T>
ForwardIterator remove(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    const T& value);

template<class ForwardIterator, class Predicate>
constexpr ForwardIterator remove_if(ForwardIterator first, ForwardIterator last, Predicate pred);

template<class ExecutionPolicy, class ForwardIterator, class Predicate>
ForwardIterator remove_if(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    Predicate pred);

template<class InputIterator, class OutputIterator, class T>
constexpr OutputIterator remove_copy(InputIterator first, InputIterator last, OutputIterator result, const T& value);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class T>
ForwardIterator2 remove_copy(ExecutionPolicy&& exec, // see 28.4.5

```

```

        ForwardIterator1 first, ForwardIterator1 last,
        ForwardIterator2 result, const T& value);

template<class InputIterator, class OutputIterator, class Predicate>
constexpr OutputIterator remove_copy_if(InputIterator first, InputIterator last, OutputIterator result, Predicate pred);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class Predicate>
ForwardIterator2 remove_copy_if(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first, ForwardIterator1 last,
    ForwardIterator2 result, Predicate pred);

// 28.6.9, unique
template<class ForwardIterator>
constexpr ForwardIterator unique(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class BinaryPredicate>
constexpr ForwardIterator unique(ForwardIterator first, ForwardIterator last, BinaryPredicate pred);

template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator unique(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last);

template<class ExecutionPolicy, class ForwardIterator, class BinaryPredicate>
ForwardIterator unique(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    BinaryPredicate pred);

template<class InputIterator, class OutputIterator>
constexpr OutputIterator unique_copy(InputIterator first, InputIterator last, OutputIterator result);

template<class InputIterator, class OutputIterator, class BinaryPredicate>
constexpr OutputIterator unique_copy(InputIterator first, InputIterator last, OutputIterator result, BinaryPredicate pred);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 unique_copy(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first, ForwardIterator1 last,
    ForwardIterator2 result);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
ForwardIterator2 unique_copy(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first, ForwardIterator1 last,
    ForwardIterator2 result, BinaryPredicate pred);

// 28.6.10, reverse
template<class BidirectionalIterator>
constexpr void reverse(BidirectionalIterator first, BidirectionalIterator last);

template<class ExecutionPolicy, class BidirectionalIterator>
void reverse(ExecutionPolicy&& exec, // see 28.4.5
    BidirectionalIterator first, BidirectionalIterator last);

template<class BidirectionalIterator, class OutputIterator>
constexpr OutputIterator reverse_copy(BidirectionalIterator first, BidirectionalIterator last, OutputIterator result);

template<class ExecutionPolicy, class BidirectionalIterator, class ForwardIterator>
ForwardIterator reverse_copy(ExecutionPolicy&& exec, // see 28.4.5
    BidirectionalIterator first,
    BidirectionalIterator last,
    ForwardIterator result);

// 28.6.11, rotate
template<class ForwardIterator>
constexpr ForwardIterator rotate(ForwardIterator first, ForwardIterator middle, ForwardIterator last);

template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator rotate(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first,
    ForwardIterator middle,
    ForwardIterator last);

template<class ForwardIterator, class OutputIterator>
constexpr OutputIterator rotate_copy(ForwardIterator first, ForwardIterator middle, ForwardIterator last, OutputIterator result);

template<class ExecutionPolicy, class ForwardIterator, class ForwardIterator>
ForwardIterator rotate_copy(
    ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator middle,
    ForwardIterator last, ForwardIterator result);

// 28.6.12, sample
template<class PopulationIterator, class SampleIterator, class Distance, class UniformRandomNumberGenerator>
SampleIterator sample(PopulationIterator first, PopulationIterator last,
    SampleIterator out, Distance n,
    UniformRandomNumberGenerator&& g);

// 28.6.13, shuffle
template<class RandomAccessIterator, class UniformRandomNumberGenerator>
void shuffle(RandomAccessIterator first, RandomAccessIterator last, UniformRandomNumberGenerator&& g);

// 28.7.4, partitions
template <class InputIterator, class Predicate>
constexpr bool is_partitioned(InputIterator first, InputIterator last, Predicate pred);

template <class ExecutionPolicy, class ForwardIterator, class Predicate>
bool is_partitioned(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last, Predicate pred);

template<class ForwardIterator, class Predicate>
constexpr ForwardIterator partition(ForwardIterator first, ForwardIterator last, Predicate pred);

template<class ExecutionPolicy, class ForwardIterator, class Predicate>
ForwardIterator partition(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first,
    ForwardIterator last,
    Predicate pred);

template<class BidirectionalIterator, class Predicate>
BidirectionalIterator stable_partition(BidirectionalIterator first, BidirectionalIterator last, Predicate pred);

template<class ExecutionPolicy, class BidirectionalIterator, class Predicate>

```

```

BidirectionalIterator stable_partition(ExecutionPolicy&& exec, // see 28.4.5
    BidirectionalIterator first,
    BidirectionalIterator last,
    Predicate pred);

template <class InputIterator, class OutputIterator1, class OutputIterator2, class Predicate>
constexpr pair<OutputIterator1, OutputIterator2>
    partition_copy(InputIterator first, InputIterator last, OutputIterator1 out_true, OutputIterator2 out_false, Predicate pred);

template <class ExecutionPolicy, class ForwardIterator, class ForwardIterator1, class ForwardIterator2, class Predicate>
pair<ForwardIterator1, ForwardIterator2>
partition_copy(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    ForwardIterator1 out_true, ForwardIterator2 out_false,
    Predicate pred);

template<class ForwardIterator, class Predicate>
constexpr ForwardIterator partition_point(ForwardIterator first, ForwardIterator last, Predicate pred);

// 28.7, sorting and related operations
// 28.7.1, sorting
template<class RandomAccessIterator>
constexpr void sort(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void sort(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class ExecutionPolicy, class RandomAccessIterator>
void sort(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void sort(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last,
    Compare comp);

template<class RandomAccessIterator>
void stable_sort(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
void stable_sort(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class ExecutionPolicy, class RandomAccessIterator>
void stable_sort(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void stable_sort(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last,
    Compare comp);

template<class RandomAccessIterator>
constexpr void partial_sort(RandomAccessIterator first, RandomAccessIterator middle, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void partial_sort(RandomAccessIterator first, RandomAccessIterator middle, RandomAccessIterator last, Compare comp);

template<class ExecutionPolicy, class RandomAccessIterator>
void partial_sort(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first,
    RandomAccessIterator middle,
    RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void partial_sort(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first,
    RandomAccessIterator middle,
    RandomAccessIterator last, Compare comp);

template<class InputIterator, class RandomAccessIterator>
constexpr RandomAccessIterator partial_sort_copy(InputIterator first, InputIterator last, RandomAccessIterator result_first, RandomAccessIterator result_last);

template<class InputIterator, class RandomAccessIterator, class Compare>
constexpr RandomAccessIterator partial_sort_copy(InputIterator first, InputIterator last, RandomAccessIterator result_first, RandomAccessIterator result_last, Compare comp);

template<class ExecutionPolicy, class ForwardIterator, class RandomAccessIterator>
RandomAccessIterator partial_sort_copy(
    ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    RandomAccessIterator result_first,
    RandomAccessIterator result_last);

template<class ExecutionPolicy, class ForwardIterator, class RandomAccessIterator, class Compare>
RandomAccessIterator partial_sort_copy(
    ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    RandomAccessIterator result_first,
    RandomAccessIterator result_last,
    Compare comp);

template<class ForwardIterator>
constexpr bool is_sorted(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class Compare>
constexpr bool is_sorted(ForwardIterator first, ForwardIterator last, Compare comp);

template<class ExecutionPolicy, class ForwardIterator>
bool is_sorted(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last);

template<class ExecutionPolicy, class ForwardIterator, class Compare>
bool is_sorted(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    Compare comp);

template<class ForwardIterator>
constexpr ForwardIterator is_sorted_until(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class Compare>
constexpr ForwardIterator is_sorted_until(ForwardIterator first, ForwardIterator last, Compare comp);

```



```

template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator is_sorted_until(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last);

template<class ExecutionPolicy, class ForwardIterator, class Compare>
ForwardIterator is_sorted_until(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    Compare comp);

// 28.7.2, Nth element
template<class RandomAccessIterator>
constexpr void nth_element(RandomAccessIterator first, RandomAccessIterator nth, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void nth_element(RandomAccessIterator first, RandomAccessIterator nth, RandomAccessIterator last, Compare comp);

template<class ExecutionPolicy, class RandomAccessIterator>
void nth_element(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator nth,
    RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void nth_element(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator nth,
    RandomAccessIterator last, Compare comp);

// 28.7.3, binary search
template<class ForwardIterator, class T>
constexpr ForwardIterator lower_bound(ForwardIterator first, ForwardIterator last, const T& value);

template<class ForwardIterator, class T, class Compare>
constexpr ForwardIterator lower_bound(ForwardIterator first, ForwardIterator last, const T& value, Compare comp);

template<class ForwardIterator, class T>
constexpr ForwardIterator upper_bound(ForwardIterator first, ForwardIterator last, const T& value);

template<class ForwardIterator, class T, class Compare>
constexpr ForwardIterator upper_bound(ForwardIterator first, ForwardIterator last, const T& value, Compare comp);

template<class ForwardIterator, class T>
constexpr pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first, ForwardIterator last, const T& value);

template<class ForwardIterator, class T, class Compare>
constexpr pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first, ForwardIterator last, const T& value, Compare comp);

template<class ForwardIterator, class T>
constexpr bool binary_search(ForwardIterator first, ForwardIterator last, const T& value);

template<class ForwardIterator, class T, class Compare>
constexpr bool binary_search(ForwardIterator first, ForwardIterator last, const T& value, Compare comp);

// 28.7.5, merge
template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator merge(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result);

template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
constexpr OutputIterator merge(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result, Compare comp);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class ForwardIterator>
ForwardIterator merge(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2,
    ForwardIterator result);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class ForwardIterator, class Compare>
ForwardIterator merge(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2,
    ForwardIterator result, Compare comp);

template<class BidirectionalIterator>
void inplace_merge(BidirectionalIterator first, BidirectionalIterator middle, BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
void inplace_merge(BidirectionalIterator first, BidirectionalIterator middle, BidirectionalIterator last, Compare comp);

template<class ExecutionPolicy, class BidirectionalIterator>
void inplace_merge(ExecutionPolicy&& exec, // see 28.4.5
    BidirectionalIterator first,
    BidirectionalIterator middle,
    BidirectionalIterator last);

template<class ExecutionPolicy, class BidirectionalIterator, class Compare>
void inplace_merge(ExecutionPolicy&& exec, // see 28.4.5
    BidirectionalIterator first,
    BidirectionalIterator middle,
    BidirectionalIterator last, Compare comp);

// 28.7.6, set operations
template<class InputIterator1, class InputIterator2>
constexpr bool includes(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2);

template<class InputIterator1, class InputIterator2, class Compare>
constexpr bool includes(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, Compare comp);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
bool includes(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class Compare>
bool includes(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2, Compare comp);

template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator set_union(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result);

template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
constexpr OutputIterator set_union(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result, Compare comp);

```


[illegible]

```

template<class RandomAccessIterator>
constexpr RandomAccessIterator is_heap_until(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr RandomAccessIterator is_heap_until(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class ExecutionPolicy, class RandomAccessIterator>
RandomAccessIterator is_heap_until(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
RandomAccessIterator is_heap_until(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last,
    Compare comp);

// 28.7.8, minimum and maximum
template<class T> constexpr const T& min(const T& a, const T& b);

template<class T, class Compare>
constexpr const T& min(const T& a, const T& b, Compare comp);

template<class T>
constexpr T min(initializer_list<T> t);

template<class T, class Compare>
constexpr T min(initializer_list<T> t, Compare comp);

template<class T> constexpr const T& max(const T& a, const T& b);

template<class T, class Compare>
constexpr const T& max(const T& a, const T& b, Compare comp);

template<class T>
constexpr T max(initializer_list<T> t);

template<class T, class Compare>
constexpr T max(initializer_list<T> t, Compare comp);

template<class T> constexpr pair<const T&, const T&> minmax(const T& a, const T& b);

template<class T, class Compare>
constexpr pair<const T&, const T&> minmax(const T& a, const T& b, Compare comp);

template<class T>
constexpr pair<T, T> minmax(initializer_list<T> t);

template<class T, class Compare>
constexpr pair<T, T> minmax(initializer_list<T> t, Compare comp);

template<class ForwardIterator>
constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last)

template<class ForwardIterator, class Compare>
constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last, Compare comp);

template<class ExecutionPolicy, class ForwardIterator>
constexpr ForwardIterator min_element(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last);

template<class ExecutionPolicy, class ForwardIterator, class Compare>
constexpr ForwardIterator min_element(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    Compare comp);

template<class ForwardIterator>
constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last)

template<class ForwardIterator, class Compare>
constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last, Compare comp);

template<class ExecutionPolicy, class ForwardIterator>
constexpr ForwardIterator max_element(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last);

template<class ExecutionPolicy, class ForwardIterator, class Compare>
constexpr ForwardIterator max_element(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    Compare comp);

template<class ForwardIterator>
constexpr pair<ForwardIterator, ForwardIterator>
    minmax_element(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class Compare>
constexpr pair<ForwardIterator, ForwardIterator>
    minmax_element(ForwardIterator first, ForwardIterator last, Compare comp);

template<class ExecutionPolicy, class ForwardIterator>
constexpr pair<ForwardIterator, ForwardIterator> minmax_element(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last);

template<class ExecutionPolicy, class ForwardIterator, class Compare>
constexpr pair<ForwardIterator, ForwardIterator> minmax_element(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last, Compare comp);

// 28.7.9, bounded value
template<class T>
constexpr const T& clamp(const T& v, const T& lo, const T& hi);

template<class T, class Compare>
constexpr const T& clamp(const T& v, const T& lo, const T& hi, Compare comp);

// 28.7.10, lexicographical comparison
template<class InputIterator1, class InputIterator2>
constexpr bool lexicographical_compare(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2);

template<class InputIterator1, class InputIterator2, class Compare>
constexpr bool lexicographical_compare(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, Compare comp);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>

```

```
bool lexicographical_compare(
    ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class Compare>
bool lexicographical_compare(
    ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2,
    Compare comp);

// 28.7.11, permutations
template<class BidirectionalIterator>
constexpr bool next_permutation(BidirectionalIterator first, BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
constexpr bool next_permutation(BidirectionalIterator first, BidirectionalIterator last, Compare comp);

template<class BidirectionalIterator>
constexpr bool prev_permutation(BidirectionalIterator first, BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
constexpr bool prev_permutation(BidirectionalIterator first, BidirectionalIterator last, Compare comp);

}
```

B. Modifications to remaining parts of "28 Algorithms library" [algorithms] (all sections except "Header <algorithm> synopsis" and "28.8 C library algorithms" [alg.c.library])

Note for editor: All the functions marked with `constexpr` in previous paragraph of this document must be accordingly marked with `constexpr` in detailed algorithm description. For shortness only modifications to "28.5.1 All of" [alg.all_of] are shown in this paper.

28.5.1 All of [alg.all_of]

```
template <class InputIterator, class Predicate>
constexpr bool all_of(InputIterator first, InputIterator last, Predicate pred);

template <class ExecutionPolicy, class ForwardIterator, class Predicate>
bool all_of(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
    Predicate pred);
```

Returns: true if [first,last) is empty or if `pred(*i)` is true for every iterator `i` in the range [first,last), and false otherwise.

Complexity: At most `last - first` applications of the predicate.

C. Modifications to "23.2 Utility components" [utility]

```
// 23.2.3, swap:
template<class T>
constexpr void swap(T& a, T& b) noexcept(see below );

template <class T, size_t N>
constexpr void swap(T (&a)[N], T (&b)[N]) noexcept(noexcept(swap(*a, *b)));

// 23.2.4, exchange:
template <class T, class U=T>
constexpr T exchange(T& obj, U&& new_val);
```

D. Modifications to "23.2.3 swap" [utility.swap]

```
template<class T> constexpr void swap(T& a, T& b) noexcept(see below );
```

Remark: This function shall not participate in overload resolution unless `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true. The expression inside `noexcept` is equivalent to:
`is_nothrow_move_constructible_v<T> && is_nothrow_move_assignable_v<T>`

Requires: Type `T` shall be `MoveConstructible` (Table 20) and `MoveAssignable` (Table 22).
Effects: Exchanges values stored in two locations.

```
template<class T, size_t N>
constexpr void swap(T (&a)[N], T (&b)[N]) noexcept(is_nothrow_swappable_v<T>);
```

Remarks: This function shall not participate in overload resolution unless `is_swappable_v<T>` is true.

Requires: `a[i]` shall be swappable with (20.5.3.2) `b[i]` for all `i` in the range `[0,N)`.
Effects: As if by `swap_ranges(a, a + N, b)`.

E. Modifications to "23.2.4 exchange" [utility.exchange]

```
template <class T, class U=T> constexpr T exchange(T& obj, U&& new_val);
```

Effects: Equivalent to:
`T old_val = std::move(obj);`
`obj = std::forward<U>(new_val);`
`return old_val;`

F. Feature-testing macro

For the purposes of SG10, we recommend the feature-testing macro name `__cpp_lib_constexpr_algorithms`.

V. Revision History

Revision 2:
- Do not add <code>constexpr</code> to the <code>ExecutionPolicy</code> overloads. - Updated wording to use N4687.

Revision 1:

- Dropped all the modifications for the `<cstring>` header.
- Dropped `constexpr` for `stable_sort`, `stable_partition`, `inplace_merge`, `shuffle` and `sample`.
- Updated wording to use N4582.
- Applied `constexpr` to the `ExecutionPolicy` overloads and algorithms added since N4567.
- Oulu voting:

```
Should some part of this target C++17?
SF F N A SA
3 3 3 3 1

Should the non-parallel parts be forwarded for C++Next?
SF F N A SA
6 7 0 0 0
```

Revision 0:

- Initial proposal
- Jacksonville voting:

```
Approve of adding constexpr to the algorithms?
SF F N A SA
9 4 3 0 0
```

```
Approve of adding constexpr to algorithms that need some compiler intrinsics, pending some compiler vendors approving?
SF F N A SA
2 8 5 1 0
```

Resolution: `needs_updated_proposal`

VI. References

[N4687] Working Draft, Standard for Programming Language C++. Available online at www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4687.pdf

[P0031R0] A Proposal to Add `constexpr` Modifiers to `reverse_iterator`, `move_iterator`, `array` and `Range` Access. Available online at www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0031r0.html

[rhalbersma] Proof of concept for some functions. Available online at <https://bitbucket.org/rhalbersma/xstd/src/42553df6107623c71163f104b6f3cc550c245b4b/include/xstd/algorithm.hpp> and <https://bitbucket.org/rhalbersma/xstd/src/42553df6107623c71163f104b6f3cc550c245b4b/include/xstd/utility.hpp>

[Boost.Algorithm] `constexpr` modifiers for Boost Algorithm library. Available online at <https://github.com/boostorg/algorithm/pull/13>

[Discussion] A call to discuss `asm` in `constexpr` and `constexpr` `<algorithm>`. Available online at <https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/9sTJWsOpptE>

[P0202R0] Add `constexpr` Modifiers to Functions in `<algorithm>` and `<cstring>` Headers. Available online at <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0202r0.html>

VII. Acknowledgements

Walter E. Brown provided numerous comments, corrections, and suggestions for this proposal.